

Relatório Técnico

Implementação Parcial de um Compilador para Subconjunto da Linguagem
C

Igor de Matos da Rosa

Matrícula: 20103930

Disciplina: DEC0004-09655 (20261) – Compiladores

Universidade Federal de Santa Catarina (UFSC)

Professor: Alison Roberto Panisson

Araranguá, 2026

Sumário

1	Introdução	3
2	Fundamentação Teórica e Relação com o Conteúdo da Disciplina	3
2.1	Unidade 1 – Introdução à Construção de Compiladores	3
2.2	Unidade 2 – Análise Léxica	4
2.3	Unidade 3 – Análise Sintática	4
2.4	Unidade 4 – Análise Semântica	4
3	Especificação do Trabalho	5
4	Arquitetura da Implementação	5
4.1	Fluxo de Processamento	6
5	Detalhamento da Análise Léxica	6
5.1	Tokens Reconhecidos	6
5.2	Relação com a Teoria	6
6	Detalhamento da Análise Sintática	6
6.1	Principais Produções Gramaticais	6
6.2	Relação com a Teoria	7
7	Detalhamento da Análise Semântica	7
7.1	Ações Semânticas Implementadas	7
7.2	Tabela de Símbolos	7
7.3	Verificação de Tipos	8
8	Testes e Validação	8
8.1	Relatório de Execução	8
9	Considerações Finais	9

1 Introdução

Este relatório descreve a implementação parcial de um compilador desenvolvida como Trabalho Final da disciplina DEC0004-09655 (20261) – Compiladores, ministrada na Universidade Federal de Santa Catarina. O objetivo da atividade consiste em integrar, em um único artefato de software, as fases de **análise léxica**, **análise sintática** e **análise semântica**, conforme especificado no enunciado da tarefa avaliativa.

A solução foi construída de forma incremental: a análise léxica constitui evolução do Trabalho 1 (T1); a análise sintática, do Trabalho 2 (T2); e a análise semântica e o relatório consolidado de execução foram incorporados nesta entrega final. A linguagem-alvo é um **subconjunto restrito de C**, suficiente para demonstrar as estruturas exigidas: declarações de variáveis, atribuições, expressões aritméticas, estruturas condicionais (simples e aninhadas) e laços de repetição (**while** e **for**).

A implementação utiliza a biblioteca **PLY** (*Python Lex-Yacc*), ferramenta didática amplamente empregada na disciplina para a construção de analisadores léxicos e sintáticos a partir de expressões regulares e gramáticas livres de contexto.

2 Fundamentação Teórica e Relação com o Conteúdo da Disciplina

A organização deste trabalho segue a estrutura curricular disponibilizada no ambiente virtual da disciplina, articulando teoria e prática nas unidades estudadas.

2.1 Unidade 1 – Introdução à Construção de Compiladores

Conforme os materiais introdutórios da disciplina, um compilador transforma um programa-fonte em um programa-alvo equivalente, percorrendo fases sequenciais de análise e síntese. Neste trabalho, implementam-se as três primeiras fases da **análise**:

1. **Análise léxica** – particionamento do fluxo de caracteres em tokens;
2. **Análise sintática** – verificação da estrutura conforme uma gramática;
3. **Análise semântica** – verificação de regras de contexto não expressas pela gramática.

As fases de geração e otimização de código (Unidade 5) não foram implementadas, pois extrapolam o escopo do trabalho final, que exige uma *implementação parcial*.

2.2 Unidade 2 – Análise Léxica

A análise léxica realiza o reconhecimento de **lexemas** por meio de um **autômato finito**, usualmente especificado por expressões regulares. No modelo teórico, o analisador léxico remove comentários e espaços em branco, identifica palavras reservadas, literais, identificadores e operadores, encaminhando tokens ao analisador sintático.

Na implementação, cada categoria léxica é definida por uma função `t_*` ou por uma expressão regular associada a um terminal PLY. Palavras reservadas como `int`, `if`, `while` e `for` são mapeadas para tokens distintos de identificadores genéricos (ID). Operadores compostos (`==`, `<=`, `>=`, `!=`, `++`, `-`) possuem regras declaradas **antes** dos operadores simples, evitando conflitos de *prefixo mais longo* (*maximal munch*).

2.3 Unidade 3 – Análise Sintática

A análise sintática verifica se a sequência de tokens pode ser derivada a partir da gramática da linguagem. A disciplina aborda analisadores **top-down** (LL) e **bottom-up** (LR). O PLY Yacc implementa um analisador do tipo **LALR(1)**, subclasse dos analisadores LR, adequada a gramáticas de linguagens de programação.

Os conceitos de **conjuntos First e Follow**, bem como a resolução de ambiguidades, manifestam-se na gramática implementada. Destaca-se a ambiguidade clássica do `if-then-else`, resolvida mediante declaração de precedência:

```
precedence = (  
    ('nonassoc', 'IFX'),  
    ('nonassoc', 'ELSE'),  
)
```

A regra condicional com `ELSE` recebe menor precedência que o `IF` sem `ELSE`, associando o `else` ao `if` mais próximo — regra usual em linguagens como C.

A gramática reconhece, entre outras estruturas, o programa principal `int main()`, blocos compostos, declarações, atribuições, expressões com precedência aritmética, condicionais aninhados e laços `while/for`.

2.4 Unidade 4 – Análise Semântica

A análise semântica verifica propriedades que a gramática não captura: existência de identificadores, compatibilidade de tipos e consistência de escopos. A disciplina distingue **atributos sintetizados** (calculados nos filhos e propagados ao pai) e **atributos herdados** (propagados do pai aos filhos).

Nesta implementação, adota-se um esquema predominantemente **S-atribuído**: cada nó de expressão sintetiza seu tipo em `p[0]`, permitindo verificações nas produções de atribuição, declaração e estruturas de controle. A **tabela de símbolos** materializa o

conceito de **escopo léxico**, implementado como pilha de tabelas hash: cada bloco `{}` abre um novo escopo e o encerramento do bloco restaura o escopo anterior.

3 Especificação do Trabalho

O enunciado do Trabalho Final estabelece os seguintes requisitos:

- análise sintática capaz de validar atribuições, operações aritméticas, condicionais (simples e aninhadas) e laços de repetição;
- análise semântica com pelo menos três ações semânticas;
- reconhecimento do programa-exemplo fornecido no enunciado;
- entrega de código-fonte, `README.txt` e relatório teórico-prático.

O programa de referência do enunciado é reproduzido abaixo:

```
1 int main()
2 {
3     int a, b, resultado;
4     a = 4;
5     b = 6;
6     if(a >= 10){
7         resultado = a - b;
8     }else{
9         resultado = a + b;
10    }
11 }
```

4 Arquitetura da Implementação

O projeto está organizado em módulos com responsabilidade única:

Tabela 1: Estrutura de arquivos do projeto

Arquivo	Responsabilidade
<code>compilador_c.py</code>	Analisador léxico, gramática e orquestração
<code>tabela_simbolos.py</code>	Gerenciamento de escopos e declarações
<code>semantica.py</code>	Ações semânticas e verificação de tipos
<code>relatorio.py</code>	Consolidação do relatório de execução
Testes/	Programas de validação
<code>README.txt</code>	Instruções de execução

4.1 Fluxo de Processamento

O compilador parcial executa o seguinte pipeline ao receber um arquivo `.c`:

1. **Fase léxica:** leitura do arquivo e geração da sequência de tokens;
2. **Fase sintática:** aplicação da gramática LALR(1) sobre os tokens;
3. **Fase semântica:** execução das ações associadas às produções durante o *parsing* (esquema ad hoc com atributos sintetizados);
4. **Relatório final:** apresentação consolidada dos resultados das três fases e veredito de validade do programa.

A execução é parametrizável:

```
1 python3 compilador_c.py [--lex | --sint] <arquivo.c>
```

5 Detalhamento da Análise Léxica

5.1 Tokens Reconhecidos

O analisador léxico reconhece palavras reservadas, identificadores, literais (inteiros, ponto flutuante, caracteres e *strings*), operadores aritméticos e relacionais, delimitadores e comentários (linha e bloco). Espaços, tabulações e retornos de carro são ignorados; quebras de linha atualizam o contador de linhas para diagnósticos de erro.

5.2 Relação com a Teoria

Cada regra léxica corresponde a um padrão regular que pode ser representado por um AFD. A ferramenta PLY constrói internamente o autômato a partir das expressões fornecidas, materializando na prática o conteúdo da Unidade 2 sobre geradores de analisadores léxicos e a transição de especificação declarativa para reconhecedor executável.

6 Detalhamento da Análise Sintática

6.1 Principais Produções Gramaticais

A gramática é centrada no não terminal `programa`, derivando para a função `main` e seu bloco. Destacam-se as produções:

```

programa          : funcao_principal semicolon_opcional
funcao_principal  : INT MAIN LPAREN parametro_main RPAREN bloco
comando           : declaracao SEMICOLON
                  | atribuicao SEMICOLON
                  | comando_condicional
                  | comando_repeticao
                  | bloco
comando_condicional : IF LPAREN condicao RPAREN comando %prec IFX
                  | IF LPAREN condicao RPAREN comando ELSE comando
comando_repeticao   : WHILE LPAREN condicao RPAREN comando
                  | FOR LPAREN inicio_for SEMICOLON
                      condicao_opcional
                      SEMICOLON atualizacao_for RPAREN comando
expressao          : expressao PLUS termo
                  | expressao MINUS termo
                  | termo

```

A hierarquia $\text{expressao} \rightarrow \text{termo} \rightarrow \text{potencia} \rightarrow \text{fator}$ implementa precedência de operadores: adição/subtração, multiplicação/divisão e potenciação, respectivamente.

6.2 Relação com a Teoria

O analisador LALR(1) construído pelo Yacc realiza *shift* e *reduce* conforme a tabela de ações estudada na Unidade 3. Cada função p_* representa uma produção da gramática; a redução bem-sucedida confirma que o trecho analisado pertence à linguagem definida.

7 Detalhamento da Análise Semântica

7.1 Ações Semânticas Implementadas

Foram implementadas **seis** ações semânticas, superando o mínimo de três exigido pelo enunciado:

7.2 Tabela de Símbolos

A classe `TabelaSimbolos` mantém uma pilha de dicionários. Ao encontrar `{`, um novo escopo é empilhado; ao encontrar `}`, o escopo é desempilhado. A operação de busca percorre os escopos do mais interno ao mais externo, reproduzindo a regra de resolução de nomes de C.

Tabela 2: Ações semânticas implementadas

Ação	Fundamento teórico
Inserção na tabela de símbolos	Escopo léxico e declaração
Verificação de uso antes da declaração	Regra de contexto
Detecção de redeclaração no mesmo escopo	Política de escopos aninhados
Compatibilidade de tipos em atribuições	Checagem de tipos
Tipagem de expressões aritméticas/relacionais	Atributos sintetizados
Validação de condições em estruturas de controle	Restrição semântica de <code>if/while/for</code>

7.3 Verificação de Tipos

Os tipos primitivos considerados são: `int`, `float`, `double`, `char` e `string`. Expressões aritméticas aplicam promoção numérica ($char \leq int \leq float \leq double$). Operadores relacionais exigem operandos numéricos e produzem tipo `int`, analogamente à convenção de C.

8 Testes e Validação

Foram elaborados programas de teste em `Testes/`, classificados em válidos (espera-se compilação sem erros semânticos) e inválidos (espera-se diagnóstico semântico específico).

Tabela 3: Casos de teste

Arquivo	Resultado	Objetivo
<code>ExemploEnunciado.c</code>	Válido	Programa do enunciado
<code>Teste1.c</code>	Válido	Condicionais aninhados
<code>Teste2.c</code>	Válido	Laço <code>while</code>
<code>Teste3.c</code>	Válido	<code>for</code> , <code>while</code> e <code>if</code>
<code>Erro_VariavelNaoDeclarada.c</code>	Inválido	Uso sem declaração
<code>Erro_Redeclaracao.c</code>	Inválido	Redeclaração no escopo
<code>Erro_TipoIncompativel.c</code>	Inválido	Atribuição <code>int ← string</code>
<code>Erro_CondicaoInvalida.c</code>	Inválido	Condição não numérica

8.1 Relatório de Execução

Ao término da análise, o módulo `relatorio.py` imprime um relatório estruturado contendo: (i) lista de tokens com linha, tipo e valor; (ii) estruturas sintáticas reconhecidas; (iii) tabela de símbolos e ações semânticas executadas; e (iv) veredito final (VÁLIDO ou INVÁLIDO). Esse relatório materializa, de forma operacional, a integração das três fases analíticas estudadas na disciplina.

9 Considerações Finais

O trabalho consolidou os conhecimentos das Unidades 2, 3 e 4 da disciplina de Compiladores, transformando conceitos teóricos — autômatos finitos, gramáticas livres de contexto, analisadores LR, tabelas de símbolos e esquemas de atributos — em um compilador parcial funcional.

As limitações do subconjunto implementado (ausência de geração de código, tipos compostos, funções definidas pelo usuário e pré-processador) delimitam o escopo de uma *implementação parcial*, coerente com o enunciado. Como continuidade natural do estudo, a Unidade 5 poderia estender o projeto com geração de código intermediário ou código objeto.

Referências

1. PANISSON, A. R. *Material didático da disciplina DEC0004-09655 (20261) – Compiladores*. UFSC, 2026. Unidades 1 a 4 (Introdução, Análise Léxica, Análise Sintática e Análise Semântica).
2. AHO, A. V.; LAM, M. S.; SETHI, R.; ULLMAN, J. D. *Compiladores: Princípios, Técnicas e Ferramentas*. 2. ed. São Paulo: Pearson, 2008.
3. BEAZLEY, D. M. *PLY (Python Lex-Yacc)*. Disponível em: <https://www.dabeaz.com/ply/>. Acesso em: jun. 2026.